

Programmieren für Kids

JavaScript für Kids

Modul 5

Dein großes Spiel



Trainer: Wolfgang Schedl

Kurz zurückgeblickt

Du hast jetzt **alle** Bausteine des Programmierens:

- Variablen und Funktionen
- Entscheidungen mit `if/else`
- Zufall und Schleifen
- Text und Farben auf der Seite

Heute bringst du alles zusammen, ohne einen einzigen neuen Befehl.

Das machst du in diesem Modul

- zwei große Spiele bauen: ein **Kampf-Spiel** und das **Klick-Imperium**
- deine Seiten **schön** machen (ein bisschen CSS)
- dein eigenes Projekt **vorführen**

Der große Abschluss. Zeig, was du kannst.



Kapitel 1

Deine großen Spiele



Projekt

Held gegen Monster



Jetzt öffnen: [5a-held-gegen-monster](#)

in VS Code – oder auf der Kursseite



So ist deine Vorlage aufgebaut

```
<h2 id="held">Held: 100 Leben</h2>
<h2 id="monster">Monster: 100 Leben</h2>
<p id="text">Drück auf Angriff!</p>
<button onclick="angriff()">Angriff!</button>
```

Zwei Anzeigen (Held und Monster), eine Nachricht und ein Knopf, der `angriff()` ruft.

♥ Die Lebenspunkte

Beide starten mit 100 Leben. Eine Funktion zeigt sie an:

```
let heldLeben = 100
let monsterLeben = 100

function zeige() {
  document.getElementById("held").innerHTML = "Held: " + heldLeben
    + " Leben"
  document.getElementById("monster").innerHTML = "Monster: " +
    monsterLeben + " Leben"
}
```



Der Angriff: Zufallsschaden

Bei jedem Klick treffen beide einander mit zufälligem Schaden:

```
function angriff() {  
  const schaden = Math.floor(Math.random() * 20) + 5  
  monsterLeben = monsterLeben - schaden  
  
  const monsterSchaden = Math.floor(Math.random() * 20) + 5  
  heldLeben = heldLeben - monsterSchaden  
  
  zeige()  
}
```

`Math.random` macht jeden Kampf anders, `+ 5` sorgt für mindestens 5 Schaden.

Wer gewinnt?

Am Ende jedes Angriffs prüfen wir, ob jemand keine Leben mehr hat:

```
if (monsterLeben <= 0) {  
    document.getElementById("text").innerHTML = "Du hast  
        gewonnen!"  
} else if (heldLeben <= 0) {  
    document.getElementById("text").innerHTML = "Das Monster  
        gewinnt!"  
}
```

Diese Prüfung kommt ans Ende der `angriff`-Funktion.

Übung: dein Kampf-Spiel



PROJEKT

ca. 15 Minuten

- Bring den Kampf zum Laufen (Angriff und Gewinner-Prüfung)
- Ändere die Namen: Ritter gegen Drache, oder was du magst

★ **Bonus:** Baue einen „Heiltrank“-Knopf, der dem Helden zufällig Leben zurückgibt.



Zeig dein Kampf-Spiel!

Held: 60 Leben

Monster: 25 Leben

Du machst 18 Schaden!

Angriff!



Projekt

Das Klick-Imperium



Jetzt öffnen: [5b-klick-imperium](#)

in VS Code – oder auf der Kursseite

Was ist das Spiel?

Ein Klick-Spiel, das süchtig macht:

- du klickst und sammelst Punkte
- mit den Punkten kaufst du Upgrades
- Upgrades machen jeden Klick stärker
- und du sammelst noch schneller

Das Beste: du brauchst nichts Neues, nur alles, was du schon kannst.

Klicken bringt Punkte

```
let punkte = 0

function klick() {
  punkte = punkte + 1
  document.getElementById("punkte").textContent = punkte
}
```

Ein Knopf ruft `klick()`, und die Punkte steigen. Das kennst du schon.

Punkte pro Klick

Der Trick fürs Wachsen: wie viel ein Klick wert ist, merken wir in einer zweiten Kiste.

```
let punkte = 0
let proKlick = 1

function klick() {
  punkte = punkte + proKlick
  document.getElementById("punkte").textContent = punkte
}
```

Solange proKlick 1 ist, ändert sich nichts. Ein Upgrade macht daraus mehr.

Alles anzeigen

Bald ändern sich mehrere Zahlen. Eine Funktion frischt sie alle auf einmal auf:

```
function zeigeAlles() {  
    document.getElementById("punkte").textContent = punkte  
    document.getElementById("proKlick").textContent = proKlick  
}
```

Nach jeder Änderung rufst du `zeigeAlles()` auf. So bleibt der Code ordentlich.

Übung: der Klick zählt



ÜBUNG

ca. 12 Minuten

- Bring den Klick-Knopf zum Laufen (mit `proKlick`)
- Schreibe `zeigeAlles` und ruf es nach jedem Klick auf

★ **Bonus:** Starte mit `proKlick = 5`, dann bringt jeder Klick gleich 5 Punkte.

Kaufen mit if/else

Jetzt der Laden. Ein Upgrade, aber nur, wenn du es dir leisten kannst:

```
let preisMaus = 10

function kaufeMaus() {
  if (punkte >= preisMaus) {
    punkte = punkte - preisMaus
    proKlick = proKlick + 1
    zeigeAlles()
  } else {
    alert("Nicht genug Punkte!")
  }
}
```

Steigende Preise

Damit es spannend bleibt, wird das Upgrade nach jedem Kauf teurer. Nur eine Zeile mehr:

```
punkte = punkte - preisMaus  
proKlick = proKlick + 1  
preisMaus = preisMaus * 2  
zeigeAlles()
```

Aus 10 wird 20, dann 40, dann 80. Jedes Upgrade ist stärker verdient als das letzte.

Ein zweites Upgrade

Ein zweites Upgrade nach demselben Muster, nur mit anderen Zahlen:

```
let preisFabrik = 100

function kaufeFabrik() {
  // gleiches Muster wie kaufeMaus,
  // aber mit preisFabrik und proKlick + 10
}
```

▶ Teste dein Imperium

Bevor du weiterbaust: Speichern und ausprobieren.

- Steigen die Punkte beim Klicken?
- Kannst du ein Upgrade kaufen, wenn du genug hast?
- Wird jeder Klick danach stärker?

Immer ein Stück bauen und testen, dann das nächste.

Fehler-Detektive



FINDE DEN FEHLER

Dieses Upgrade ist gratis, man wird stärker, ohne zu zahlen. Warum?

```
function kaufeMaus() {  
  if (punkte >= preisMaus) {  
    proKlick = proKlick + 1  
    zeigeAlles()  
  }  
}
```

Tipp: Wird der Preis irgendwo abgezogen?

Übung: dein eigener Laden



PROJEKT

ca. 15 Minuten

- Bring Klicken und beide Upgrades zum Laufen
- Erfinde ein eigenes Upgrade (Name, Preis, Stärke)

★ **Bonus:** Baue ein drittes, sehr teures Upgrade, z.B. einen „Roboter“ für 1000 Punkte.

Zeig dein Imperium!

128

Punkte • pro Klick: 3

Klick!

Maus

Fabrik



Kapitel 2

Mach es schön: CSS



Was ist CSS?

HTML ist das Gerüst. **CSS** ist das Aussehen, die Kleidung deiner Seite.

- Farben, Größen, Schriften, Abstände
- bisher hast du CSS fertig bekommen
- jetzt darfst du es **selbst** ändern

CSS steht im `<style>`-Bereich deiner Vorlage.



Wo steht das CSS?

Ganz oben in deiner Datei, im `<style>`-Bereich:

```
<head>
  <style>
    /* ===== HIER kommt dein CSS ===== */

  </style>
</head>
```

HTML sagt, was da ist. CSS im `<style>` sagt, *wie* es aussieht.



Farben ändern

Im `<style>`-Bereich sagst du, wie etwas aussehen soll:

```
body {  
  background-color: lightyellow;  
}
```

Erst der Name (body = die ganze Seite), dann in den Klammern die Regeln.
Probier `lightblue`, `pink`, `black`.

A Schrift und Größe

```
h1 {  
  color: hotpink;  
  font-size: 40px;  
  text-align: center;  
}
```

color ist die Textfarbe, font-size die Größe, text-align: center rückt es in die Mitte.

Knöpfe hübsch machen

```
button {  
  padding: 10px 20px;  
  border-radius: 10px;  
  background-color: skyblue;  
}
```

padding macht den Knopf größer, border-radius rundet die Ecken ab.



Viele Farben zum Ausprobieren

Du kannst einfach Farb-Namen benutzen:

red orange gold lightgreen
skyblue hotpink purple black

Probier verschiedene aus, bis dir deine Seite gefällt.

Übung: mach deine Seite schön



ÜBUNG

ca. 12 Minuten

Nimm eines deiner Projekte und verändere im `<style>`-Bereich:

- die Hintergrundfarbe der Seite
- die Farbe und Größe der Überschrift

★ **Bonus:** Mach deine Knöpfe bunt und rund.



Projekt

Dein eigenes Projekt

★ Wähle dein Projekt

Such dir aus, woran du weiterbauen willst:

- die Stimmungs-Maschine oder die Wahrsager-Kugel
- Schere, Stein, Papier oder das Zahlen-Rätsel
- das Klick-Imperium
- das Kampf-Spiel Held gegen Monster
- oder etwas ganz Eigenes



Mach es zu deinem

- füge eigene Inhalte hinzu (Fragen, Upgrades, Antworten ...)
- mach es mit CSS schön
- bau ein neues Feature ein, das du dir wünschst

Frag, wenn du Hilfe brauchst. Trau dich, Dinge auszuprobieren.

Zeig es her!

Jeder zeigt kurz sein Projekt:

- Was hast du gebaut?
- Was war am schwierigsten?
- Was gefällt dir am besten?



Das hast du alles gelernt

- **Modul 1:** mit dem Computer reden, Variablen, deine erste Seite
- **Modul 2:** Funktionen und Knöpfe, die Seite reagiert
- **Modul 3:** Zufall und Entscheidungen, zwei Spiele
- **Modul 4:** Schleifen, Dinge viele Male tun
- **Modul 5:** deine großen Spiele und dein eigener Stil

☰ Deine wichtigsten Befehle

let / const	=	etwas merken
function name()	=	ein Rezept bauen
if / else	=	entscheiden
for / while	=	wiederholen
Math.random()	=	würfeln
getElementById	=	eine Box holen



Als Nächstes: Modul 6

Deine Spiele können noch mehr:

Mit einem **Timer** passieren Dinge **von selbst** – perfekt für Reaktions- und Geschicklichkeitsspiele.

Weiter geht's mit Modul 6: Timer.